

INDEX

Sr No.	Practical Topic	Date	Sign
1	Implement a C program to identify keywords and identifiers using finite automata.		
2	Implement a C program to identify whether the production is left recursive or not and eliminate left recursion if it is applicable.		
3	Implement a C program to remove left factoring.		
4	Write a lex program to identify numbers, words and other characters and generate tokens for each.		
5	Write a lex program to identify all occurrences of “LDRP” and replace it with “COLLEGE”.		
6	Write a lex program to display the length of each word.		
7	Write a lex program to convert the lowercase first letter of the string to uppercase and uppercase first letter of the string to lowercase.		
8	Write a lex program to count the number of characters, words and lines in the given input.		
9	Write a lex program to add line numbers to every line of an input file.		
10	Write a lex program that reads the numbers and add 3 to the numbers if the number is divisible by 7.		
11	Write a lex program to remove empty lines.		
12	Write a lex program to identify words followed by punctuation marks.		
13	Write a lex program to display the comments from a given input file.		
14	Write a lex program to identify all the lexemes from the input file that follow the given RE. Provide the RE and input file as command line arguments.		
15	Write a lex program that will replace the word “Hello” with “ldrp” if the line starts with the letter ‘a’ and with “college” if it starts with ‘b’.		
16	Generate a lexer for C program.		
17	Write a Yacc program for desktop calculator with ambiguous grammar.		
18	Implement pass-I of a two pass assembler		

Practical:-1**Aim: Implement a C program to identify keywords and identifiers using finite automata.**

```

#include<stdio.h>
#include<string.h>
#include<stdlib.h>

void removeduplicate();
void final();
int Isiden(char ch);
int Isop(char ch);
int Isdel(char ch);
int Iskey(char * str);
void removeduplicate();
char op[8]={'+', '-', '*', '/', '=', '<', '>', '%'};
char del[8]={'}', '{', ',', '(', ')', '[', ']', ';'};
char *key[]={"int", "void", "main", "char", "float"};

//char *operato[]={"+", "-", "/", "*", "<", ">", "=", "%", "<=", ">=", "++"};

int idi=0, idj=0, k, opi=0, opj=0, deli=0, uqdi=0, uqidi=0, uqoperi=0, kdi=0, liti=0, ci=0;
int uqdeli[20], uqopi[20], uqideni[20], l=0, j;
char uqdel[20], uqiden[20][20], uqop[20][20], keyword[20][20];
char iden[20][20], oper[20][20], delem[20], litral[20][20], lit[20], constant[20][20];
void lexanalysis(char *str)
{
    int i=0;
    while(str[i]!='\0')
    {

```

```
if(Isiden(str[i]))    //for identifiers
{
    while(Isiden(str[i]))
    {
        iden[idi][idj++]=str[i++];
    }
    iden[idi][idj]='\0';
    idi++;idj=0;
}
else
if(str[i]=="")    //for literals
{
    lit[l++]=str[i];
    for(j=i+1;str[j]!="";j++)
    {
        lit[l++]=str[j];
    }
    lit[l++]=str[j];lit[l]='\0';
    strcpy(litral[liti++],lit);
    i=j+1;
}
else
if(Isop(str[i]))    // for operators
{
    while(Isop(str[i]))
    {
        oper[opi][opj++]=str[i++];
```

```
    }
    oper[opi][opj]='\0';
    opi++;opj=0;
}
else
if(Isdel(str[i]))    //for delemeters
{
    while(Isdel(str[i]))
    {
        delem[deli++]=str[i++];
    }
}
else
{
    i++;
}
}
removeduplicate();
final();
}

int Isiden(char ch)
{
    if(isalpha(ch)||ch=='_'||isdigit(ch)||ch=='.')
    return 1;
    else
    return 0;
}
```

```
int Isop(char ch)
{
    int f=0,i;
    for(i=0;i<8&&!f;i++)
    {
        if(ch==op[i])
            f=1;
    }
    return f;
}

int Isdel(char ch)
{
    int f=0,i;
    for(i=0;i<8&&!f;i++)
    {
        if(ch==del[i])
            f=1;
    }
    return f;
}

int Iskey(char * str)
{
    int i,f=0;
    for(i=0;i<5;i++)
    {
        if(!strcmp(key[i],str))
            f=1;
    }
}
```

```
    }  
    return f;  
}  
void removeduplicate()  
{  
    int i,j;  
    for(i=0;i<20;i++)  
    {  
        uqdeli[i]=0;  
        uqopi[i]=0;  
        uqideni[i]=0;  
    }  
    for(i=1;i<deli+1;i++) //removing duplicate delemeters  
    {  
        if(uqdeli[i-1]==0)  
        {  
            uqdel[uqdi++]=delem[i-1];  
            for(j=i;j<deli;j++)  
            {  
                if(delem[i-1]==delem[j])  
                    uqdeli[j]=1;  
            }  
        }  
    }  
}  
for(i=1;i<idi+1;i++) //removing duplicate identifiers  
{  
    if(uqideni[i-1]==0)
```

```

{
    strcpy(uqiden[uqidi++],iden[i-1]);
    for(j=i;j<idi;j++)
    {
        if(!strcmp(iden[i-1],iden[j]))
            uqideni[j]=1;
    }
} }

for(i=1;i<opi+1;i++) //removing duplicate operators
{
    if(uqopi[i-1]==0)
    {
        strcpy(uqop[uqoperi++],oper[i-1]);
        for(j=i;j<opi;j++)
        {
            if(!strcmp(oper[i-1],oper[j]))
                uqopi[j]=1;
        }
    } } }

void final()
{
    int i=0;
    idi=0;
    for(i=0;i<uqidi;i++)
    {
        if(Iskey(uqiden[i])) //identifying keywords
            strcpy(keyword[kdi++],uqiden[i]);
    }
}

```

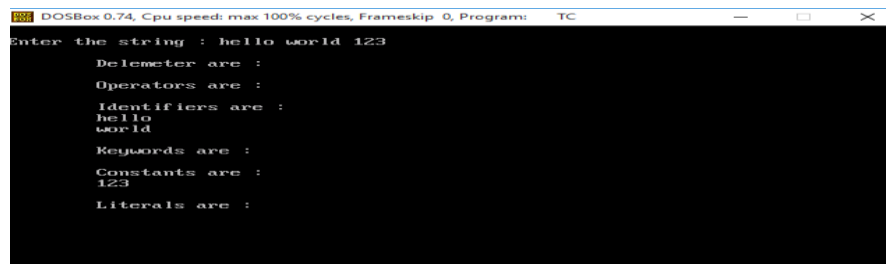
```
    else
    if(isdigit(uqiden[i][0])) //identifying constants
    strcpy(constant[ci++],uqiden[i]);
    else
    strcpy(iden[idi++],uqiden[i]);
}

// printing the outputs
printf("\n\tDelemeter are : \n");
for(i=0;i<uqdi;i++)
printf("\t%c\n",uqdel[i]);
printf("\n\tOperators are : \n");
for(i=0;i<uqoperi;i++)
{
    printf("\t");
    puts(uqop[i]);
}
printf("\n\tIdentifiers are : \n");
for(i=0;i<idi;i++)
{
    printf("\t");
    puts(iden[i]);
}
printf("\n\tKeywords are : \n");
for(i=0;i<kdi;i++)
{
    printf("\t");
    puts(keyword[i]); }
```

```
printf("\n\tConstants are :\n");
for(i=0;i<ci;i++)
{
    printf("\t");
    puts(constant[i]);
}
printf("\n\tLiterals are :\n");
for(i=0;i<liti;i++)
{
    printf("\t");
    puts(litral[i]);
} }

void main()
{
    char str[50];
    clrscr();
    printf("\nEnter the string : ");
    scanf("%[^\\n]c",str);
    lexanalysis(str);

    getch();
}
```

Output:

```
DOSBox 0.74, Cpu speed: max 100% cycles, Frameskip 0, Program: TC
Enter the string : hello world 123
Delimiter are :
Operators are :
Identifiers are :
hello
world
Keywords are :
Constants are :
123
Literals are :
```

PRACTICAL-2**AIM: Write a C program to eliminate left recursion from a production.****main.c**

```
#include<stdio.h>

#include<string.h>

#define SIZE 10

int main () {

char non_terminal;

char beta,alpha;

int num;

int i;

char production[10][SIZE];

int index=3; /* starting of the string following "->" */

printf("Enter Number of Production : ");

scanf("%d",&num);

printf("Enter the grammar as E->E-A :\n");

for(i=0;i<num;i++){

scanf("%s",production[i]);

}

for(i=0;i<num;i++){
```

```
printf("\nGRAMMAR :: %s",production[i]);
non_terminal=production[i][0];
if(non_terminal==production[i][index]) {
alpha=production[i][index+1];
printf(" is left recursive.\n");
while(production[i][index]!=0 && production[i][index]!='|') {
index++; }
if(production[i][index]!=0) {
beta=production[i][index+1];
printf("Grammar without left recursion:\n");
printf("%c->%c%c",non_terminal,beta,non_terminal);
printf("\n%c\'->%c%c\'|^\n",non_terminal,alpha,non_terminal);
}
else
printf(" can't be reduced\n");
}
else
printf(" is not left recursive.\n");
index=3;
}
return 0;
}
```

OUTPUT

```
1  #include<stdio.h>
2  #include<string.h>
3  #define SIZE 10
4  int main () {
5      char non_terminal;
6      char beta,alpha;
7      int num;
8      int i;
9      char production[10][SIZE];
10     int index=3; /* starting of the string following "->" */
11     printf("Enter Number of Production : ");
12     scanf("%d",&num);
13     printf("Enter the grammar as E->E-A :\n");
14     for(i=0;i<num;i++){
15         scanf("%s",production[i]);
16     }
17     for(i=0;i<num;i++){
18         printf("\nGRAMMAR :: %s",production[i]);
19         non_terminal=production[i][0];
20         if(non_terminal==production[i][index]) {
21             alpha=production[i][index+1];
22             printf(" is left recursive.\n");
23             while(production[i][index]!=0 && production[i][index]!='|') {
24                 index++;
25             }
26             if(production[i][index]!=0) {
27                 beta=production[i][index+1];
28                 printf("Grammar without left recursion:\n");
29                 printf("%c->%c%c\n",non_terminal,beta,non_terminal);
30                 printf("\n\n");
31             }
32         }
33     }
34 }
```

leftRecursion

Enter Number of Production : 1
Enter the grammar as E->E-A :
E->E|A

GRAMMAR :: E->E|A is left recursive.
Grammar without left recursion:
E->AE
E->E|A

Process returned 0 (0x0) execution time : 9.842 s
Press ENTER to continue.

Logs & others

Code::Blocks Search results Build log Build messages Debugger

nothing to be done (all items are up-to-date).

----- Run: Debug in leftRecursion (compiler: GNU GCC Compiler)-----

Checking for existence: /home/akash/workspace/Codeblocks/LeftRecursion/bin/Debug/leftRecursion
Executing: xterm -T leftRecursion -e /usr/bin/cb_console_runner LD_LIBRARY_PATH=SLD_LIBRARY_PATH: /home/akash/workspace/Codeblocks/LeftRecursion/bin/Debug/leftRecursion (in /home/akash/workspace/Codeblocks/LeftRecursion/.)

PRACTICAL-3

AIM: Write a C program to remove left factoring to a production.

main.c

```
#include<stdio.h>

#include<string.h>

int main()

{

char gram[20],part1[20],part2[20],modifiedGram[20],newGram[20],tempGram[20];

int i,j=0,k=0,l=0,pos;

printf("Enter Production : A->");

gets(gram);

for(i=0;gram[i]!='|';i++,j++)

part1[j]=gram[i];

part1[j]='\0';

for(j=++i,i=0;gram[j]!='\0';j++,i++)

part2[i]=gram[j];

part2[i]='\0';

for(i=0;i<strlen(part1)||i<strlen(part2);i++)

{

if(part1[i]==part2[i])

{

modifiedGram[k]=part1[i];

k++;
```

```

pos=i+1;

}

}

for(i=pos,j=0;part1[i]!='\0';i++,j++){

newGram[j]=part1[i];

}

newGram[j++]='|';

for(i=pos;part2[i]!='\0';i++,j++){

newGram[j]=part2[i];

}

modifiedGram[k]='X';

modifiedGram[++k]='\0';

newGram[j]='\0';

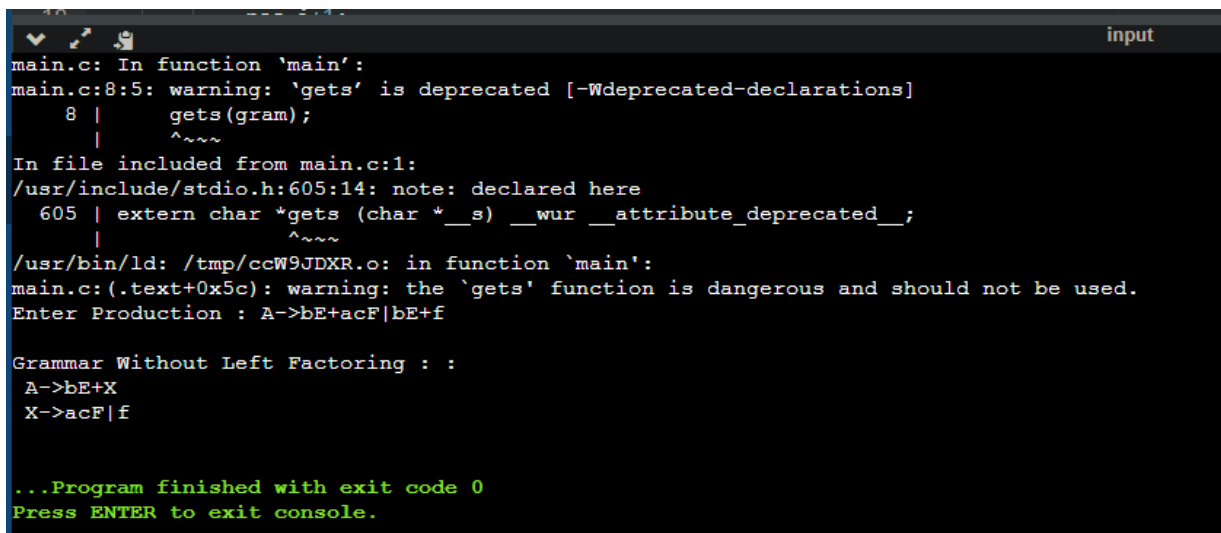
printf("\n A->%s",modifiedGram);

printf("\n X->%s\n",newGram);

}

```

OUTPUT



```

main.c: In function 'main':
main.c:8:5: warning: 'gets' is deprecated [-Wdeprecated-declarations]
    8 |     gets(gram);
      |     ^~~~
In file included from main.c:1:
/usr/include/stdio.h:605:14: note: declared here
   605 | extern char *gets (char *__s) __wur __attribute_deprecated__;
      |           ^~~~
/usr/bin/ld: /tmp/ccW9JDXR.o: in function 'main':
main.c:(.text+0x5c): warning: the 'gets' function is dangerous and should not be used.
Enter Production : A->bE+acF|bE+f

Grammar Without Left Factoring : :
A->bE+X
X->acF|f

...Program finished with exit code 0
Press ENTER to exit console.

```

PRACTICAL-4**AIM: Implement a lex program to identify keywords, identifiers.****TRY.1**

```
%{
```

```
#include<stdio.h>
```

```
%}
```

```
%%
```

```
int |
```

```
float |
```

```
char |
```

```
double |
```

```
while |
```

```
static |
```

```
for |
```

```
do |
```

```
break |
```

```
continue |
```

```
else |
```

```
goto      {printf("\n%s is a Keyword",yytext);}
```

```
[a-zA-Z_][a-zA-Z0-9_]*      {printf("\n%s is a Variable",yytext);}
```

```
[0-9]* {printf("\n%s is a Number",yytext);}
```

```
#. *      {printf("\n%s is a PREPROCESSOR DIRECTIVE",yytext);}
```

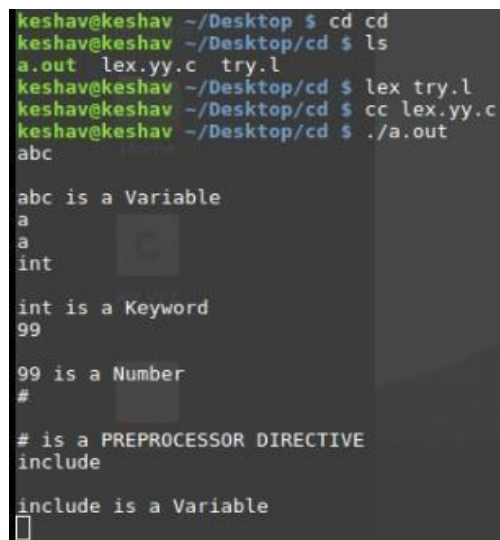
```
%%
```

```
int main()
{
    yylex();

    return 0;
}
```

```
int yywrap()
{
    return 1;
}
```

OUTPUT



```
keshav@keshav ~/Desktop $ cd cd
keshav@keshav ~/Desktop/cd $ ls
a.out  lex.yy.c  try.l
keshav@keshav ~/Desktop/cd $ lex try.l
keshav@keshav ~/Desktop/cd $ cc lex.yy.c
keshav@keshav ~/Desktop/cd $ ./a.out
abc
abc is a Variable
a
a
int
int is a Keyword
99
99 is a Number
#
# is a PREPROCESSOR DIRECTIVE
include
include is a Variable
█
```

PRACTICAL-5**AIM: Write a lex program to replace all occurrences of “ldrp” with “college”.****LDRP.L**

```
%{  
  
#include<stdio.h>  
  
%}  
  
%%  
  
ldrp {printf("College");}  
  
%%  
  
int main()  
{  
  
    yylex();  
  
    return 0;  
}  
  
int yywrap()  
{  
  
    return 1;  
}
```

OUTPUT

```
File Edit View Search Terminal Help
keshav@keshav ~ $ cd Desktop
keshav@keshav ~/Desktop $ cd cd
keshav@keshav ~/Desktop/cd $ lex ldrp.l
keshav@keshav ~/Desktop/cd $ cc lex.yy.c -o ldrp
keshav@keshav ~/Desktop/cd $ ./ldrp
ldrp
College
ldrp - itr
College - itr
keshav ldrp college
keshav College college
█
ld ldrp
yylval:
return
```

PRACTICAL-6**AIM: Write a lex program to display length of each word.****length.l**

l [a-zA-Z]

d [0-9]

o [!@#\$%^&()_+=|"<>,;?\.:]

%%

({l}*({l}|{d}|{o}))* {printf("%s %ld",yytext,yylen);}

%%

int main()

{

yylex();

return 0;

}

int yywrap()

{

return 1;

```
}
```

OUTPUT

```
keshav@keshav ~/Desktop/cd
File Edit View Search Terminal Help
keshav@keshav ~ $ cd Desktop
keshav@keshav ~/Desktop $ cd cd
keshav@keshav ~/Desktop/cd $ ls
a.out  comments.l  hello  k1.txt  k2.txt  k3.txt  length.l  lex.yy.c  punct  punct.l  remove_empty  remove_empty.l  try1  try.l
keshav@keshav ~/Desktop/cd $ lex length.l
keshav@keshav ~/Desktop/cd $ cc lex.yy.c -o length
keshav@keshav ~/Desktop/cd $ ./length
keshav
keshav 6

```

PRACTICAL-7**AIM: Write a lex program to change the case of first letter of every word.****caps.l**

%{

int i;

%}

l[a-zA-Z]

%%

({} {} *)

{

for(i=0;i<yyleng;i++)

{

if(i==0 && yytext[i]>96) printf("%c",yytext[i]- 32);

else

printf("%c",(yytext[i]));

}

}

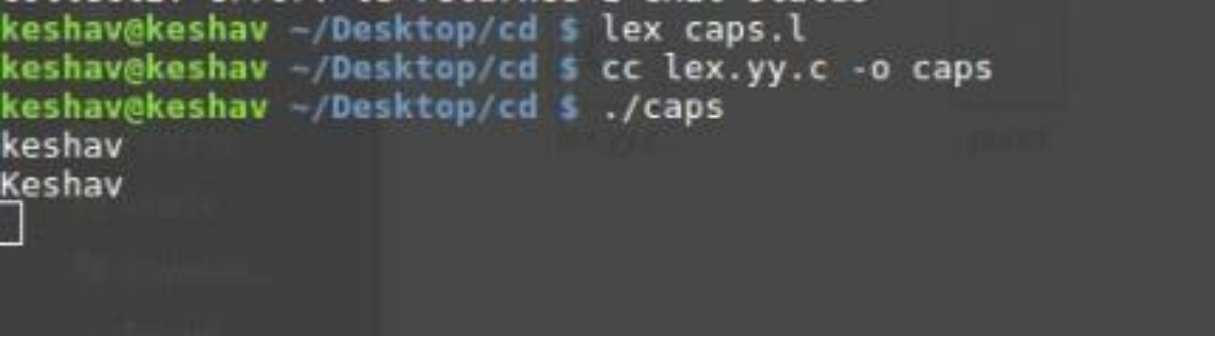
%%

int main()

{

```
    yylex();  
  
    return 0;  
  
}  
  
int yywrap()  
{  
    return 1;  
}
```

OUTPUT



```
keshav@keshav ~/Desktop/cd $ lex caps.l  
keshav@keshav ~/Desktop/cd $ cc lex.yy.c -o caps  
keshav@keshav ~/Desktop/cd $ ./caps  
keshav  
Keshav  
□
```

PRACTICAL-8

AIM: Write a lex program to count the number of characters, words and lines in the given input.

```
%{
#include <stdio.h>
int charCount = 0;
int wordCount = 0;
int lineCount = 0;
}%
%%
\n { charCount++; wordCount++; lineCount++; }
[a-zA-Z]+ { charCount += yyleng; wordCount++; }
. { charCount++; }
%%
int main() {
    yylex();
    printf("Character count: %d\n", charCount);
    printf("Word count: %d\n", wordCount);
    printf("Line count: %d\n", lineCount);
    return 0;
}
```

OUTPUT:

Output:

The number of lines=3

The number of spaces=6

The number of words=9

The number of characters are=70

PRACTICAL-9**AIM: Write a lex program to add line numbers to every line of an input file.**

```
%{
#include <stdio.h>
int lineCount = 1;
%}
%%
\n { printf("%d: %s", lineCount++, yytext); }
. { printf("%d: %s", lineCount, yytext); }
%%
int main(int argc, char *argv[]) {
    if (argc != 2) {
        printf("Usage: %s <input_file>\n", argv[0]);
        return 1;
    }
    FILE *file = fopen(argv[1], "r");
    if (file == NULL) {
        printf("Error opening file.\n");
        return 1;
    }
    yyin = file;
    yylex();
    fclose(file);
    return 0;
}
```

OUTPUT:

Output:

try1.txt

hai

hello world

test file

language processor

try2.txt

1:hai

2:hello world

3:test file

4:language processor

PRACTICAL-10

AIM: Write a lex program that reads the numbers and add 3 to the numbers if the number is divisible by 7.

```
%{
#include <stdio.h>
%}
%%
[0-9]+ {
    int num = atoi(yytext);
    if (num % 7 == 0) {
        num += 3;
        printf("%d ", num);
    } else {
        printf("%d ", num);
    }
}
.|\\n { printf("%s", yytext); }
%%

int main() {
    yylex();
    return 0;
}
```

OUTPUT:

Output:

1
2
3
4
5
6
7
8
9
0

PRACTICAL-11**AIM: Write a lex program to remove empty lines.****remove_empty.l**

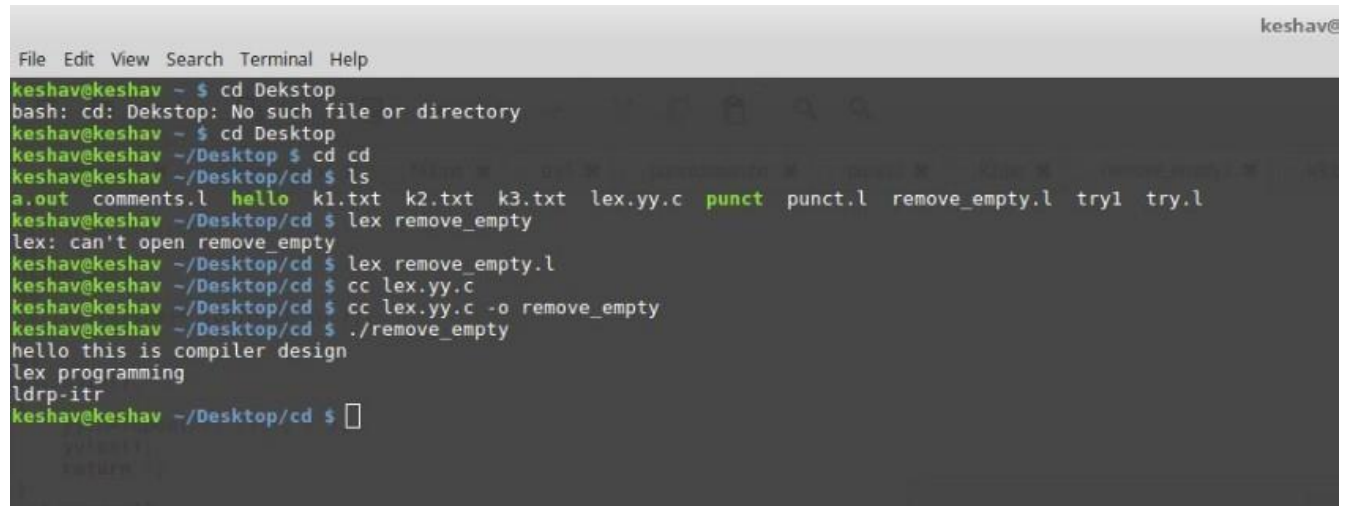
```
%{  
  
#include<stdio.h>  
  
%}  
  
nline [\n]  
  
%%  
  
{nline} {nline}+ {printf("\n");}  
  
%%  
  
int main()  
  
{  
  
    yyin=fopen("k3.txt","r");  
  
    yylex();  
  
    return 0;  
  
}  
  
int yywrap()  
  
{  
  
return 1;  
  
}
```

K3.txt

hello this is compiler design
lex programming

ldrp-itr

OUTPUT

A terminal window with a dark background and light text. The window title is 'keshav@'. The menu bar shows 'File Edit View Search Terminal Help'. The terminal content shows a series of commands and their outputs. The user attempts to change to 'Dekstop' (misspelled), then 'Desktop'. They list files in the Desktop directory, which includes 'a.out', 'comments.l', 'hello', 'k1.txt', 'k2.txt', 'k3.txt', 'lex.yy.c', 'punct', 'punct.l', 'remove_empty.l', 'try1', and 'try.l'. They then run 'lex remove_empty', which fails with 'lex: can't open remove_empty'. They then run 'lex remove_empty.l', which succeeds. Next, they compile 'lex.yy.c' with 'cc', which also succeeds. Finally, they run './remove_empty', which outputs 'hello this is compiler design', 'lex programming', and 'ldrp-itr'. The prompt returns to '~/Desktop/cd \$'.

```
keshav@keshav ~ $ cd Dekstop
bash: cd: Dekstop: No such file or directory
keshav@keshav ~ $ cd Desktop
keshav@keshav ~/Desktop $ cd cd
keshav@keshav ~/Desktop/cd $ ls
a.out  comments.l  hello  k1.txt  k2.txt  k3.txt  lex.yy.c  punct  punct.l  remove_empty.l  try1  try.l
keshav@keshav ~/Desktop/cd $ lex remove_empty
lex: can't open remove_empty
keshav@keshav ~/Desktop/cd $ lex remove_empty.l
keshav@keshav ~/Desktop/cd $ cc lex.yy.c
keshav@keshav ~/Desktop/cd $ cc lex.yy.c -o remove_empty
keshav@keshav ~/Desktop/cd $ ./remove_empty
hello this is compiler design
lex programming
ldrp-itr
keshav@keshav ~/Desktop/cd $
```

PRACTICAL-12**AIM: Write a lex program to identify words followed by punctuation marks.****punct.l**

```
%{  
  
%}  
  
letter [a-zA-Z]  
  
%%  
  
{letter}+ {printf("%s: not punctuation \n",yytext);} ;  
  
{letter}+\; {printf("%s :punctuation \n",yytext);};  
  
%%  
  
int main()  
  
{  
  
    yyin=fopen("k2.txt","r");  
  
    yylex();  
  
    return 0;  
  
}  
  
int yywrap()  
  
{  
  
return 1;  
  
}
```

K2.txt

```
hello;  
asfdas  
sadsda;  
  
sadf;
```

OUTPUT

```
File Edit View Search Terminal Help
keshav@keshav ~ $ cd Desktop
keshav@keshav ~/Desktop $ cd cd
keshav@keshav ~/Desktop/cd $ lex punct.l
keshav@keshav ~/Desktop/cd $ cc lex.yy.c
keshav@keshav ~/Desktop/cd $ cc lex.yy.c -o punct
keshav@keshav ~/Desktop/cd $ ./punct
hello; :punctuation
asfdas: not punctuation
sadfsda; :punctuation

sadf; :punctuation
keshav@keshav ~/Desktop/cd $
```

PRACTICAL-13**AIM: Write a lex program to remove the comments from the given input file.****comments.l**

%{

#include<stdio.h>

%}

%%

\W.* ;

(.\n)*.**V ;

%%

int main()

{

yyin=fopen("k1.txt","r");

yylex();

return 0;

}

int yywrap()

{

return 1;

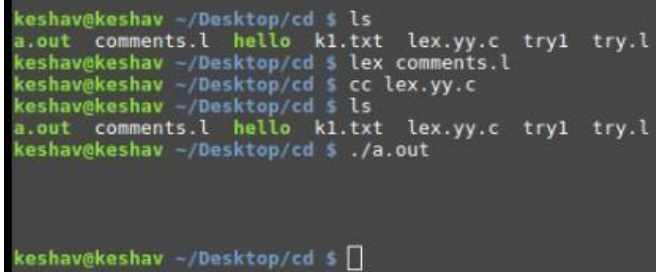
}

K1.txt

```
/* k1.txt */
```

```
/*asdfsadf*/
```

```
//empty function
```

OUTPUT

```
keshav@keshav ~/Desktop/cd $ ls
a.out  comments.l  hello  k1.txt  lex.yy.c  try1  try.l
keshav@keshav ~/Desktop/cd $ lex comments.l
keshav@keshav ~/Desktop/cd $ cc lex.yy.c
keshav@keshav ~/Desktop/cd $ ls
a.out  comments.l  hello  k1.txt  lex.yy.c  try1  try.l
keshav@keshav ~/Desktop/cd $ ./a.out

keshav@keshav ~/Desktop/cd $ █
```

PRACTICAL-14

AIM: Write a lex program to identify all the lexemes from the input file that follow the given RE. Provide the RE and input file as command line arguments.

```
%{  
  
#include <stdio.h>  
  
#include <string.h>  
  
%}  
  
%%  
  
{RE} { printf("%s\n", yytext); }  
  
.;  
  
%%  
  
int main(int argc, char *argv[]) {  
  
    if (argc != 3) {  
  
        printf("Usage: %s <regex> <input_file>\n", argv[0]);  
  
        return 1;  
  
    }  
  
    char *regex = argv[1];  
  
    char *input_file = argv[2];  
  
    FILE *file = fopen(input_file, "r");  
  
    if (file == NULL) {  
  
        printf("Error opening file.\n");  
  
        return 1;  
  
    }  
  
    yyin = file;  
  
    yy_scan_string(regex);  
  
    yylex();  
  
}
```

```
fclose(file);  
return 0;  
}
```

OUTPUT

Output:

```
$ ./a.out "[[:alpha:]]+sample[[:alpha:]]+" input.txt  
This is a sample string.
```

```
This is a sample string.  
1234567890
```

```
This  
is  
a  
sample  
string
```

PRACTICAL-15

AIM: Write a lex program that will replace the word “Hello” with “ldrp” if the line starts with the letter ‘a’ and with “college” if it starts with ‘b’.

```
%{  
#include <stdio.h>  
  
%}  
%%  
  
^a.* { gsub("Hello", "ldrp"); printf("%s\n", yytext); }  
^b.* { gsub("Hello", "college"); printf("%s\n", yytext); }  
.  
{ printf("%s\n", yytext); }  
%%  
  
int main() {  
    yylex();  
    return 0;  
}
```

OUTPUT

Output:

try1.txt

```
This is a sample string.  
Hello, world!  
Bye, bye!
```

output\

```
This is a sample string.  
ldrp, world!  
Bye, bye!
```

PRACTICAL-16**AIM: Generate a lexer for C program.****lexer.l**

```

letter [a-zA-Z]
digit [0-9]
%%
{digit}+("E"|"+"|"-"?) {digit}+)? printf("\n%s\tis real number",yytext);
{digit}+." {digit}+("E"|"+"|"-"?) {digit}+)? printf("\n%s\t is floating pt no
",yytext);
"if"|"else"|"int"|"char"|"scanf"|"printf"|"switch"|"return"|"struct"|"do"|"while"|"void
"|"for"|"float" printf("\n%s\t is keywords",yytext);
"a"|"\"|\"n"|"\"b"|"\"t"|"\"t"|"\"b"|"\"a" printf("\n%s\tis Escape sequences",yytext);
{letter}({letter}|{digit})* printf("\n%s\tis identifier",yytext);
"&&"|"<"|">"|"<="|">="|"="|"+"|"-"|"?"|"*"|"/"|"%"|"&"|"|" printf("\n%s\toperator
",yytext);
"{"}|"["]|"(")|"#"|"'"|"."|"\"|"\"|";"|"|" printf("\n%s\t is a special
character",yytext);
"%d"|"%s"|"%c"|"%f"|"%e" printf("\n%s\tis a format specifier",yytext);
%%
int yywrap()
{
return 1;
}
int main(int argc,char *argv[])
{
yyin=fopen(argv[1],"r");
yylex();
fclose(yyin);
return 0;
}

```

Cprogram.c

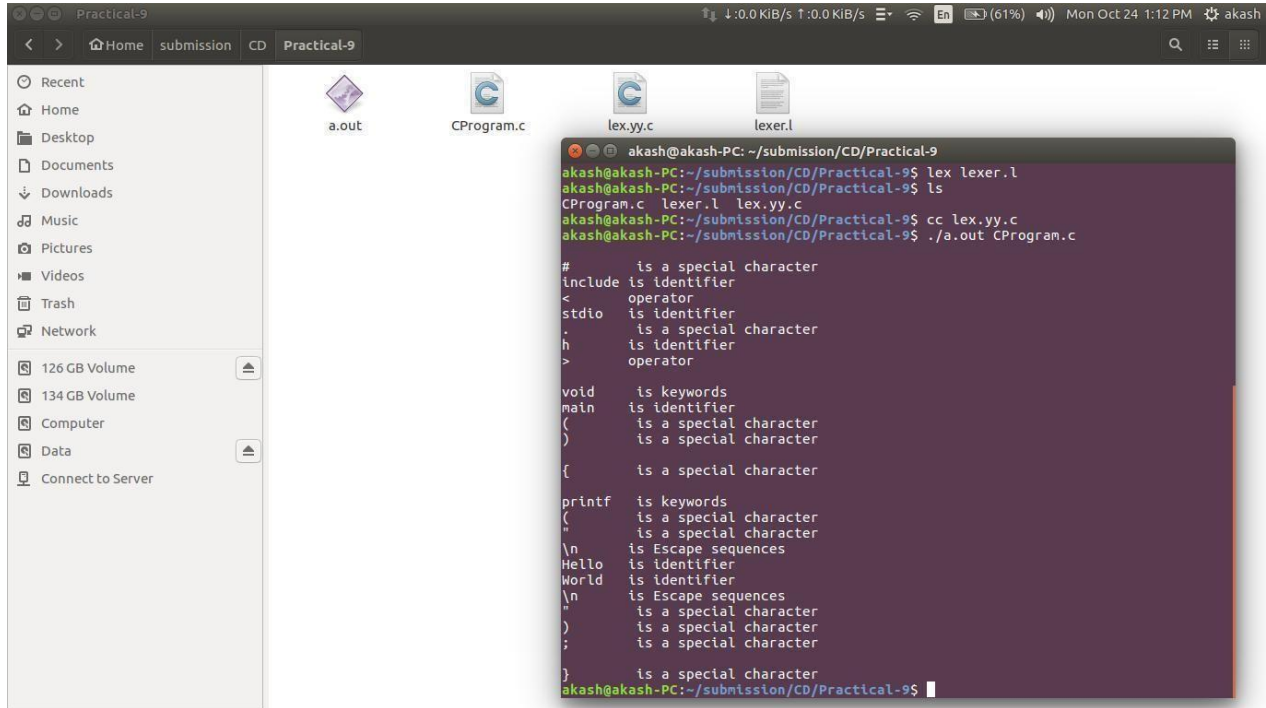
```

#include<stdio.h>
void main()

```

```
{  
printf("\nHello World\n");  
}
```

OUTPUT



The screenshot shows a Linux desktop environment with a terminal window open. The terminal displays the following commands and their outputs:

```
akash@akash-PC: ~/submission/CD/Practical-9  
akash@akash-PC:~/submission/CD/Practical-9$ lex lexer.l  
akash@akash-PC:~/submission/CD/Practical-9$ ls  
CProgram.c  lexer.l  lex.yy.c  
akash@akash-PC:~/submission/CD/Practical-9$ cc lex.yy.c  
akash@akash-PC:~/submission/CD/Practical-9$ ./a.out CProgram.c
```

The output of the program is displayed as follows:

```
# is a special character  
include is identifier  
< operator  
stdio is identifier  
. is a special character  
h is identifier  
> operator  
  
void is keywords  
main is identifier  
( is a special character  
) is a special character  
{ is a special character  
  
printf is keywords  
( is a special character  
" is a special character  
\n is Escape sequences  
Hello is identifier  
World is identifier  
\n is Escape sequences  
" is a special character  
) is a special character  
; is a special character  
  
} is a special character  
akash@akash-PC:~/submission/CD/Practical-9$
```

PRACTICAL-17**AIM: Write a Yacc program for desktop calculator with ambiguous grammar.****calc.y**

```

%{
#include <stdio.h>
#include <ctype.h>
#define YYSTYPE double /* double type for Yacc stack */
%}

%token NUMBER

%left '+' '-'
%left '*' '/'
%right UMINUS

%%

lines : lines expr '\n'      { printf("%g\n", $2); /* print result */ }
      | lines '\n'          { ; /* allow blank lines */ }
      |                     { ; /* empty string */ }
      ;

expr  : expr '+' expr        { $$ = $1 + $3; }
      | expr '-' expr        { $$ = $1 - $3; }
      | expr '*' expr        { $$ = $1 * $3; }
      | expr '/' expr        { $$ = $1 / $3; }
      | '(' expr ')'          { $$ = $2; }
      | '-' expr %prec UMINUS { $$ = - $2; }
      | NUMBER
      ;

%%

int yylex(void)
{ int c;
while ( (c = getchar()) == " ); /* skip spaces */
if ( (c == '.') || isdigit(c) ) {
ungetc(c, stdin);
scanf("%lf", &yylval); /* input a double */
return NUMBER;

```

```
}  
return c;  
}
```

y.output

0 \$accept : lines \$end

1 lines : lines expr '\n'

2 | lines '\n'

3 |

4 expr : expr '+' expr

5 | expr '-' expr

6 | expr '*' expr

7 | expr '/' expr

8 | '(' expr ')'

9 | '-' expr

10 | NUMBER

state 0

\$accept : . lines \$end (0)

lines : . (3)

. reduce 3

lines goto 1

state 1

\$accept : lines . \$end (0)

lines : lines . expr '\n' (1)

lines : lines . '\n' (2)

\$end accept

NUMBER shift 2

'-' shift 3

'\n' shift 4

(' shift 5

. error

expr goto 6

state 2

expr : NUMBER . (10)

. reduce 10

state 3

expr : '-' . expr (9)

NUMBER shift 2

'-' shift 3

(' shift 5

. error

expr goto 7

state 4

lines : lines '\n' . (2)

. reduce 2

state 5

expr : '(' . expr ')' (8)

NUMBER shift 2

'-' shift 3

(' shift 5

. error

expr goto 8

state 6

lines : lines expr . '\n' (1)

expr : expr . '+' expr (4)

expr : expr . '-' expr (5)

expr : expr . '*' expr (6)

expr : expr . '/' expr (7)

'+' shift 9

'-' shift 10

'*' shift 11

'/' shift 12

'\n' shift 13

. error

state 7

expr : expr . '+' expr (4)

expr : expr . '-' expr (5)

expr : expr . '*' expr (6)

expr : expr . '/' expr (7)
expr : '-' expr . (9)

. reduce 9

state 8

expr : expr . '+' expr (4)
expr : expr . '-' expr (5)
expr : expr . '*' expr (6)
expr : expr . '/' expr (7)
expr : '(' expr . ')' (8)

'+' shift 9
'-' shift 10
'*' shift 11
'/' shift 12
)' shift 14
. error

state 9

expr : expr '+' . expr (4)

NUMBER shift 2
'-' shift 3
'(' shift 5
. error

expr goto 15

state 10

expr : expr '-' . expr (5)

NUMBER shift 2
'-' shift 3
'(' shift 5
. error

expr goto 16

state 11

expr : expr '*' . expr (6)

NUMBER shift 2

'-' shift 3

(' shift 5

. error

expr goto 17

state 12

expr : expr '/' . expr (7)

NUMBER shift 2

'-' shift 3

(' shift 5

. error

expr goto 18

state 13

lines : lines expr '\n' . (1)

. reduce 1

state 14

expr : '(' expr ')' . (8)

. reduce 8

state 15

expr : expr . '+' expr (4)
expr : expr '+' expr . (4)
expr : expr . '-' expr (5)
expr : expr . '*' expr (6)
expr : expr . '/' expr (7)

'*' shift 11
'/' shift 12
'+' reduce 4
'-' reduce 4
'\n' reduce 4
)' reduce 4

state 16

expr : expr . '+' expr (4)
expr : expr . '-' expr (5)
expr : expr '-' expr . (5)
expr : expr . '*' expr (6)
expr : expr . '/' expr (7)

'*' shift 11
'/' shift 12
'+' reduce 5
'-' reduce 5
'\n' reduce 5
)' reduce 5

state 17

expr : expr . '+' expr (4)
expr : expr . '-' expr (5)
expr : expr . '*' expr (6)
expr : expr '*' expr . (6)
expr : expr . '/' expr (7)

. reduce 6

state 18

expr : expr . '+' expr (4)

expr : expr . '-' expr (5)

expr : expr . '*' expr (6)

expr : expr . '/' expr (7)

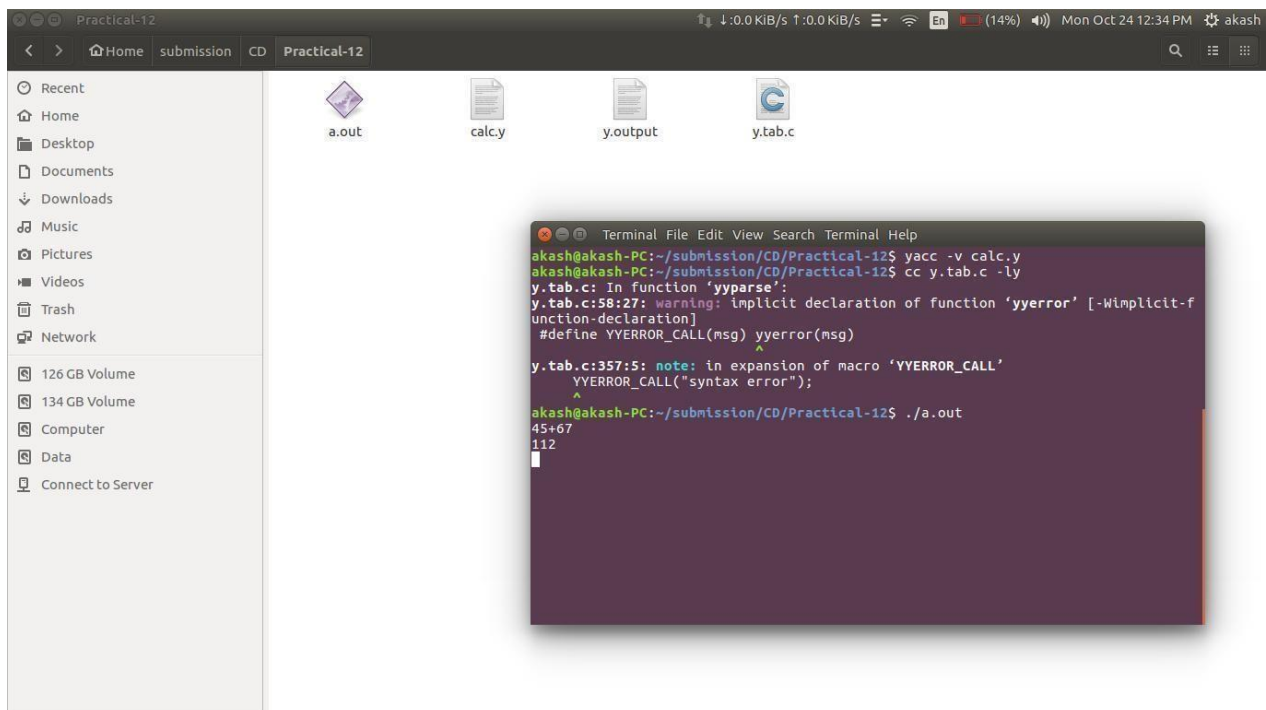
expr : expr '/' expr . (7)

. reduce 7

11 terminals, 3 nonterminals

11 grammar rules, 19 states

OUTPUT



The screenshot shows a Linux desktop environment. The file manager window displays the contents of the 'Practical-12' directory, which includes files 'a.out', 'calc.y', 'y.output', and 'y.tab.c'. A terminal window is open in the foreground, showing the following commands and output:

```
akash@akash-PC:~/submission/CD/Practical-12$ yacc -v calc.y
akash@akash-PC:~/submission/CD/Practical-12$ cc y.tab.c -ly
y.tab.c: In function 'yyparse':
y.tab.c:58:27: warning: implicit declaration of function 'yyerror' [-Wimplicit-f
unction-declaration]
#define YYERROR_CALL(msg) yyerror(msg)
                           ^
y.tab.c:357:5: note: in expansion of macro 'YYERROR_CALL'
YYERROR_CALL("syntax error");
    ^
akash@akash-PC:~/submission/CD/Practical-12$ ./a.out
45+67
112
```

PRACTICAL-18**AIM: Implement pass-I of a two pass assembler**

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAX_LINES 100
typedef struct {
    char label[10];
    char opcode[10];
    char operand[10];
} Instruction;
typedef struct {
    char label[10];
    int address;
} Symbol;
int main() {
    FILE *input = fopen("input.asm", "r");
    if (!input) {
        printf("Error: Could not open input file.\n");
        return 1;
    }
    Instruction program[MAX_LINES];
    Symbol symbolTable[MAX_LINES];
    int LOCCTR = 0;
    int symbolCount = 0;
    char line[100];
    while (fgets(line, sizeof(line), input)) {
        char *token = strtok(line, "\t\n");
        if (token != NULL) {
            strcpy(program[LOCCTR].label, token);
            token = strtok(NULL, "\t\n");
            strcpy(program[LOCCTR].opcode, token);
            token = strtok(NULL, "\t\n");
            strcpy(program[LOCCTR].operand, token);
            if (strcmp(program[LOCCTR].label, "-") != 0) {
                strcpy(symbolTable[symbolCount].label, program[LOCCTR].label);
                symbolTable[symbolCount].address = LOCCTR;
                symbolCount++;
            }
        }
    }
```

```

LOCCTR++;
}
}
fclose(input);
printf("Symbol Table:\n");
for (int i = 0; i < symbolCount; i++) {
printf("%s\t%d\n", symbolTable[i].label, symbolTable[i].address);
}
printf("\nIntermediate Code:\n");
for (int i = 0; i < LOCCTR; i++) {
printf("%d\t%s\t%s\t%s\n", i, program[i].label, program[i].opcode,
program[i].operand);
}
return 0;
}

```

OUTPUT

```

START MOV A, B
ADD A, C
SUB A, D
HLT
LOOP EQU 10

```

Output:

Symbol Table:

LOOP 10

Intermediate Code:

0	START	MOV	A, B
1	MOV	A, B	
2	ADD	A, C	
3	SUB	A, D	
4	HLT		